Massachusetts Institute of Technology

CTN Slip–Stick Piezo Motor Filter Selector Board

Comprehensive Design Report: Filter Design & Motor Measurements, Microcontroller Communication, and CAD Status

CTN Controls Team: Maanas Goel, Dr. Slawek Gras

LIGO Laboratory and Kavli Institute for Astrophysics and Space Research,
Massachusetts Institute of Technology
185 Albany Street, Cambridge, Massachusetts 02139, USA

August 23, 2025

Contents

1 Project Overview **2**

2 Requirements and Constraints **2**

3 Filter Design **3**
3.1 Intuition and Topology . . . 3
3.2 Exact sizing of C for n cascaded sections . . . 3
3.3 Settling and time-domain guidance . . . 3
3.4 Other target corners with the same R . . . 3
3.5 Component tolerances and expected spread . . . 3
3.6 Source/load interaction with motor capacitance . . . 4
3.7 Noise and thermal considerations . . . 4

4 Slip–Stick Motor Measurements **4**
4.1 What and how we measured . . . 4
4.2 Voltage-divider method for EPR (step-by-step) . . . 4
4.3 Structured measurement tables (from lab notes) . . . 4
4.4 EPR from DC Divider (Before/After 1 MΩ) . . . 6

5 Selector, LED Driver, and Sizing **6**
5.1 MOSWITCH-SPDT behavior (summary) . . . 6
5.2 NPN LED stage calculations . . . 6

6 Microcontroller Communication (Detailed) **6**
6.1 Electrical and physical . . . 6
6.2 Transport and framing . . . 6
6.3 Command protocol (quick reference) — *fixed to compile* . . . 7
6.4 Formal mini-grammar (for completeness) . . . 7
6.5 State machine . . . 7
6.6 Example terminal session . . . 7
6.7 Reference firmware (Arduino/SAMD21) . . . 7
6.8 Legacy bench scripts (two pins) . . . 8

7 Pin Mapping and Connectors **9**
7.1 Per-channel 15-pin connectors . . . 9
7.2 Control header (uChip to board) . . . 10

8 Altium CAD Status and ECO/Compile Fix **10**
8.1 Current status . . . 10
8.2 Fix checklist (do in order) . . . 10

9 Bring-Up and Test Plan **10**
9.1 Power-only checks . . . 10
9.2 Selector verification . . . 10
9.3 With motor attached . . . 11

10 Bill of Materials (condensed) **11**

11 Conclusion and Future Work **11**

Scope Confirmation

- **Filter design and measurements** (capacitance & EPR via voltage divider) are fully documented in Sections 3–4, including derivations, step-by-step procedures, worked examples, and structured data tables transcribed from our lab notes.
- **Microcontroller communication** is covered in detail in Section 6, with electrical levels, a concise ASCII protocol, a state machine, error handling, a quick-reference command table that compiles correctly, and example sessions and firmware.

1 Project Overview

We are building a **12-channel filter/selector** that sits between a controller and slip–stick piezo motors. Per channel, **Pin 1** (the useful signal/bias) can be routed either:

a) **Directly** through a fixed 30 kΩ series path, or

b) **Through a three-stage RC low-pass** (three cascaded sections with R=10 kΩ and C≈0.811 µF each), giving an overall ≈10 Hz corner.

Selection is done by a MOSFET SPDT module (*MOSWITCH-SPDT*) whose `CTRL` pin is driven by a uChip microcontroller. Each channel also has a simple BC337 NPN stage that sinks an LED to indicate "Filtered".

Why this matters. Slip–stick piezo mechanisms are largely capacitive at low frequency. We want to low-pass noisy signals and steps before they reach the motor, but we also want a *direct path* for calibration or when extra bandwidth is needed. Keeping the source-facing series resistance at 30 kΩ in both paths avoids gain jumps when switching.

2 Requirements and Constraints

- **Channel count:** 12 identical channels with identical behavior.
- **I/O:** Two 15-pin connectors per channel; Pins 2–8 and 10–15 are pass-through; Pin 9 is GND; **Pin 1 is switched/filterable**.
- **Corner frequency:** Overall −3 dB at about 10 Hz in the filtered path; alternatives 5 Hz and 1 Hz are supported by different capacitor stuffing (Table 1).
- **Constant input loading:** Direct path uses 30 kΩ series; filtered path uses three 10 kΩ sections in series ⇒ same 30 kΩ total.
- **Controller interface:** uChip (SAMD21) at 3.3 V logic, one `CTRL` per channel, USB CDC command interface.
- **Indicators:** LED per channel (on when Filtered), powered from 5 V.
- **Manufacturability:** 0805 passives, SOT-23/SOT-223/TO-92 for transistors as available, standard 2.54 mm headers for MOSWITCH modules.
- **EMI/ESD:** Series resistors on `CTRL` lines (330 Ω–1 kΩ); decoupling at MCU; optional TVS at long cable connectors.

3 Filter Design

3.1 Intuition and Topology

Each single RC section acts like a soft brake on fast changes. Cascading three sections raises attenuation *and* improves out-of-band slope to -60 dB/dec nominally, but keeps phase behavior gentle enough for quasi-static motion. To avoid level jumps when switching, both the direct and filtered paths present $30\,\mathrm{k}\Omega$ back to the source.

3.2 Exact sizing of C for n cascaded sections

For n identical first-order low-pass sections of cutoff f_1, the overall magnitude is

$$|H(j\omega)| = \Big(1 + (\omega/\omega_1)^2\Big)^{-n/2}.$$

We want $|H(j\omega_0)| = 1/\sqrt{2}$ at the overall target f_0:

$$(1 + (\omega_0/\omega_1)^2)^{n/2} = \sqrt{2} \quad \Rightarrow \quad \frac{\omega_0}{\omega_1} = \sqrt{2^{1/n} - 1}.$$

With $f_1 = 1/(2\pi RC)$, we obtain

$$\boxed{C = \frac{\sqrt{2^{1/n} - 1}}{2\pi R f_0}}.$$

For $n = 3$, $R = 10\,\mathrm{k}\Omega$, $f_0 = 10\,\mathrm{Hz}$, $\sqrt{2^{1/3} - 1} = 0.5099$, so

$$C \approx \frac{0.5099}{2\pi \cdot 10^4 \cdot 10} = 0.811\,\mu\mathrm{F} \text{ (per section)}.$$

3.3 Settling and time-domain guidance

The per-section time constant is $\tau_1 = 1/(2\pi f_1) \approx 8.1\,\mathrm{ms}$. Because the cascade behaves roughly like a third-order system, *effective* settling for step-like inputs benefits from a margin; we budget 50 ms to 100 ms after selecting FILTERED before sampling or issuing the next motion.

3.4 Other target corners with the same R

Table 1: Capacitance *per section* for different overall targets ($R = 10\,\mathrm{k}\Omega$).

Stages	C @ 10 Hz	C @ 5 Hz	C @ 1 Hz
1	1.592 µF	3.183 µF	15.915 µF
2	1.024 µF	2.049 µF	10.243 µF
3	0.811 µF	1.623 µF	8.116 µF

3.5 Component tolerances and expected spread

Assuming E12 resistors at ±1% and film capacitors at ±10%, first-order sensitivity gives $\Delta f_0/f_0 \approx \Delta C/C + \Delta R/R$. Worst case is roughly $+11\%/-11\%$ per section; for three identical sections the *overall* -3 dB point varies by a similar fraction (the exact multi-pole sensitivity is slightly smaller). If required, we can trim with a small parallel capacitor on the last section (e.g. ±100–220 nF).

3.6 Source/load interaction with motor capacitance

At low frequency the motor looks like $R_p \| \frac{1}{sC_m}$ where C_m is on the order of 50 120 nF depending on axis, and R_p is several MΩ. Because our filter corner is 10 Hz, the motor's ωC_m reactance at 10 Hz is $\sim 160\,\text{k}\Omega$ for $C_m = 100\,\text{nF}$, much larger than 30 kΩ; therefore the filter-dominant pole set by R and the on-board Cs is preserved. The DC leakage R_p only matters for bias settling and the EPR measurement; see Section 4.

3.7 Noise and thermal considerations

The 30 kΩ source resistance produces Johnson noise $e_n = \sqrt{4kTRB} \approx 0.7\,\mu\text{V}_{\text{rms}}$ over 10 Hz bandwidth at room temperature, negligible compared to motor and controller noise. LED currents (few mA) are kept off the signal net and referenced locally to ground to avoid coupling.

4 Slip–Stick Motor Measurements

4.1 What and how we measured

We characterized two aspects:

1. **AC behavior (LCR meter):** capacitance C, equivalent series resistance (ESR), and apparent leakage R at 1 kHz and 100 Hz.
2. **DC leakage (EPR):** a simple, robust **voltage-divider** measurement at steady state.

4.2 Voltage-divider method for EPR (step-by-step)

Setup. Place a known series resistor $R_1 = 1\,\text{M}\Omega$ between the bias source and the motor. Measure at DC after full settling: V_{in} *before* R_1 and V_{out} *after* R_1 (at the motor pin).

Why it works. At DC, capacitors open and the motor reduces to a resistor R_p to ground. The divider then obeys $V_{\text{out}}/V_{\text{in}} = R_p/(R_1 + R_p)$.

Solve for R_p.

$$\boxed{R_p = R_1 \frac{V_{\text{out}}}{V_{\text{in}} - V_{\text{out}}}}$$

Worked example. With $R_1 = 1\,\text{M}\Omega$, $V_{\text{in}} = 57.5\,\text{V}$, $V_{\text{out}} = 51.6\,\text{V}$,

$$R_p = 10^6 \frac{51.6}{57.5 - 51.6} = 10^6 \frac{51.6}{5.9} \approx 8.746\,\text{M}\Omega.$$

Uncertainty tips. Use a DMM with $\geq 10\,\text{M}\Omega$ input; wait $\geq 5\,\text{s}$ after a step; keep leads short; repeat both polarities and average.

4.3 Structured measurement tables (from lab notes)

Group A (translation stages)

Table 2: Channel A2 — input translation stage

Freq	C	ESR	R (leakage)	L
1 kHz	109.7 nF	122.9 Ω	16.978 kΩ	≈ 0
100 Hz	114 nF	216 Ω	900 kΩ	≈ 0

Table 3: Channel A1 — output translation stage

Freq	C	ESR	R (leakage)	L
1 kHz	113 nF	35 Ω	58 kΩ	≈ 0
100 Hz	114.7 nF	203.9 Ω	491.3 kΩ	≈ 0

Table 4: Channel A3 — translation stage

Freq	C	ESR	R (leakage)	L
1 kHz	114.2 nF	35.6 Ω	54.4 kΩ	≈ 0
100 Hz	115.9 nF	203 Ω	927 kΩ	≈ 0

Group C (tip–tilt sample mirror)

Table 5: C1 and C2 — tip–tilt sample mirror

	C1			**C2**		
Freq	C	ESR	R	C	ESR	R
1 kHz	56 nF	59.7 Ω	135.4 kΩ	57.2 nF	57.7 Ω	134.3 kΩ
100 Hz	56 nF	533 Ω	1.47 MΩ	58.1 nF	512 Ω	1.47 MΩ

Group D ("output capillars") and additional inputs

Table 6: D1 and D2 — "output capillars"

	D1			**D2**		
Freq	C	ESR	R	C	ESR	R
1 kHz	54.8 nF	57 Ω	47.9 kΩ	56.2 nF	55.8 Ω	143.6 kΩ
100 Hz	55.8 nF	506.5 Ω	1.60 MΩ	57.2 nF	496 Ω	1.56 MΩ

Table 7: C3 and D3 — input capillars

	C3			**D3**		
Freq	C	ESR	R	C	ESR	R
1 kHz	55.6 nF	56.9 Ω	144.1 kΩ	56.4 nF	55.2 Ω	144.4 kΩ
100 Hz	56.6 nF	503 Ω	1.57 MΩ	57.3 nF	490 Ω	1.57 MΩ

Group B (app string mirrors)

Table 8: B1 and B2 — app string mirrors

	B1			**B2**		
Freq	C	ESR	R	C	ESR	R
1 kHz	2.48 nF	712 Ω	712 Ω	57.4 nF	51 Ω	140 kΩ
100 Hz	3.3 nF	713 Ω	713 Ω	60.5 nF	450.5 Ω	1.54 MΩ

4.4 EPR from DC Divider (Before/After 1 MΩ)

Table 9: EPR estimates from DC divider with $R_1 = 1\,\mathrm{M\Omega}$

Channel	V_{in} [V]	V_{out} [V]	R_p [MΩ]
A1	55.27	49.74	8.995
A3	58.26	52.38	8.908
C1	55.73	48.77	7.007
C2	54.58	49.06	8.888
Example (notes)	57.50	51.60	8.746

Interpretation. C clusters around 55 nF to 60 nF for mirrors and 110 nF to 116 nF for translation axes. ESR rises when going from 1 kHz to 100 Hz, as expected. The DC leakage R_p lies in the 7 MΩ to 9 MΩ range, sufficiently large that our 30 kΩ series network dominates low-frequency behavior.

5 Selector, LED Driver, and Sizing

5.1 MOSWITCH-SPDT behavior (summary)

The module is a MOSFET H-switch acting as an SPDT: `CTRL=LOW` selects OUT-A (default), `CTRL=HIGH` selects OUT-B. Switching occurs in tens of microseconds and is silent. Observe the vendor limits for voltage/current and ensure `CTRL` is within logic spec (we drive at 3.3 V).

5.2 NPN LED stage calculations

We target a conservative LED current $I_F \approx 2\,\mathrm{mA}$. With $V_{LED} \approx 1.75\,\mathrm{V}$ (red) and $V_{CE(sat)} \approx 0.2\,\mathrm{V}$, the series resistor is

$$R_{LED} = \frac{5 - 1.75 - 0.2}{0.002} \approx 1.525\,\mathrm{k\Omega} \Rightarrow 1.5\,\mathrm{k\Omega}\ (\mathrm{E24}).$$

For the BC337 base, aiming for $\beta_{\mathrm{sat}} \sim 10$ at $I_C = 2\,\mathrm{mA}$, $I_B \sim 0.2\,\mathrm{mA}$. With $V_{BE} \approx 0.7\,\mathrm{V}$ and $V_{GPIO} = 3.3\,\mathrm{V}$,

$$R_B = \frac{3.3 - 0.7}{0.0002} \approx 13\,\mathrm{k\Omega}.$$

We use 3.3 kΩ to guarantee saturation even across part spread and temperature, while keeping GPIO current $\leq 0.8\,\mathrm{mA}$ (within SAMD21 limits). LED power is tiny: $P \approx (2\,\mathrm{to}\,3)\,\mathrm{mW}$.

6 Microcontroller Communication (Detailed)

6.1 Electrical and physical

- Logic is 3.3 V (**not** 5V-tolerant). CTRL lines: add 330 Ω–1 kΩ series near the MCU.
- Decouple MCU with 100 nF ceramics close to VCC/AVCC; add 1 µF bulk.
- Keep LED and CTRL return currents local; avoid sharing sensitive signal returns.

6.2 Transport and framing

- USB CDC serial at 115200 baud, 8-N-1.
- One ASCII command per line, newline-terminated (`\textbackslash n`).
- Responses are single lines; errors return human-readable messages.

6.3 Command protocol (quick reference) — *fixed to compile*

Table 10: ASCII command set for the 12-channel selector

Command	Meaning	Typical Response
SET,<A|B|...|L>,<DIR|FILT>	Set a single channel	OK
GET,<A|B|...|L>	Query a channel	STATE,<X>,<DIR|FILT>
ALL,<DIR|FILT>	Set all channels	OK
VER	Firmware version	VER,pxfilt,1.0

6.4 Formal mini-grammar (for completeness)

```
letter := "A" | "B" | ... | "L"
mode := "DIR" | "FILT"
cmd := "SET," letter "," mode
      | "GET," letter
      | "ALL," mode
      | "VER"
resp := "OK"
      | "STATE," letter "," mode
      | "VER,pxfilt,1.0"
      | "ERR,<reason>"
```

6.5 State machine

Each channel maintains a state {DIR, FILT}. A SET updates the state and drives the associated GPIO. After selecting FILT, firmware delays 50–100 ms before reporting ready (either inline or via a future RDY notification if we extend the protocol).

6.6 Example terminal session

```
> VER
VER,pxfilt,1.0
> GET,A
STATE,A,DIR
> SET,A,FILT
OK
> GET,A
STATE,A,FILT
> ALL,DIR
OK
```

6.7 Reference firmware (Arduino/SAMD21)

Listing 1: Firmware skeleton for 12 channels (map pins to PCB)

```
const uint8_t CH_PINS[12] = { 2,3,4,5,6,7, 8,9,10,11,12,13 }; // example mapping
enum Mode : uint8_t { DIR = 0, FILT = 1 };
Mode state[12];

void setChannel(uint8_t i, Mode m) {
  state[i] = m;
```

```
  digitalWrite(CH_PINS[i], (m==FILT) ? HIGH : LOW); // HIGH=FILTERED
  if (m==FILT) delay(60); // allow filter to settle (50-100 ms)
}

void setAll(Mode m) {
  for (uint8_t i=0;i<12;i++) digitalWrite(CH_PINS[i], (m==FILT)?HIGH:LOW);
  if (m==FILT) delay(100);
}

void setup() {
  for (uint8_t i=0;i<12;i++) pinMode(CH_PINS[i], OUTPUT);
  setAll(DIR);
  Serial.begin(115200);
}

void loop() {
  static String line;
  while (Serial.available()) {
    char c = Serial.read();
    if (c=='\n' || c=='\r') { if (line.length()) handle(line), line=""; }
    else line += c;
  }
}

int idxFromLetter(char c) {
  if ('A'<=c && c<='L') return c-'A';
  if ('a'<=c && c<='l') return c-'a';
  return -1;
}

void handle(const String& s) {
  if (s=="VER") { Serial.println("VER,pxfilt,1.0"); return; }
  int p1 = s.indexOf(',');
  if (p1<0) return;
  String op = s.substring(0,p1);
  int p2 = s.indexOf(',', p1+1);
  if (op=="ALL") { setAll(s.endsWith("FILT")?FILT:DIR); Serial.println("OK"); return;
      }
  if (op=="GET" && p2<0) {
    int i = idxFromLetter(s.charAt(p1+1));
    if (i>=0) { Serial.print("STATE,"); Serial.print((char)('A'+i));
      Serial.print(","); Serial.println((state[i]==FILT)?"FILT":"DIR"); }
    return;
  }
  if (op=="SET" && p2>0) {
    int i = idxFromLetter(s.charAt(p1+1));
    if (i>=0) { setChannel(i, s.substring(p2+1)=="FILT" ? FILT : DIR); Serial.println(
        "OK"); }
  }
}
```

6.8 Legacy bench scripts (two pins)

Listing 2: Original bench Python used during bring-up

Python host

```
import serial, time
arduino = serial.Serial(port='/dev/cu.usbmodem101', baudrate=9600, timeout=1)
```

```
def command(cmd):
    arduino.write(cmd.encode()); return arduino.readline().decode('utf-8').strip()
try:
    print("A1/A0=Pin1, B1/B0=Pin2, L1/L0=LED, Q=quit")
    while True:
        cmd = input("Enter:")
        if cmd == 'Q': break
        print("Controller Response:", command(cmd))
finally:
    arduino.close()
```

Listing 3: Original uChip-relay sketch (two outputs + LED)

uChip sketch

```
const uint8_t pins[] = {A1, A2};
void setup() {
  for (int i=0;i<sizeof(pins);i++){ pinMode(pins[i], OUTPUT); digitalWrite(pins[i], 0)
      ; }
  REG_PORT_DIRSET0 = PORT_PA15; REG_PORT_OUTSET0 = PORT_PA15; // enable 5V buck
  SerialUSB.begin(9600); pinMode(LED_BUILTIN, OUTPUT); while(!SerialUSB);
  SerialUSB.println("Setup complete, your controller says hello!");
  digitalWrite(LED_BUILTIN, 1);
}
void loop() {
  if (SerialUSB.available()){
    String s = SerialUSB.readStringUntil('\n'); s.trim();
    if (s=="A1"){ digitalWrite(pins[0],1); SerialUSB.println("Pin 1 activated."); }
    else if(s=="A0"){ digitalWrite(pins[0],0); SerialUSB.println("Pin 1 deactivated.")
        ; }
    else if(s=="B1"){ digitalWrite(pins[1],1); SerialUSB.println("Pin 2 activated.");
        }
    else if(s=="B0"){ digitalWrite(pins[1],0); SerialUSB.println("Pin 2 deactivated.")
        ; }
    else if(s=="L1"){ digitalWrite(LED_BUILTIN,1); SerialUSB.println("Good morning!");
         }
    else if(s=="L0"){ digitalWrite(LED_BUILTIN,0); SerialUSB.println("Good night!"); }
    else SerialUSB.println("Invalid command. Use A1/A0/B1/B0 or L1/L0.");
  }
}
```

7 Pin Mapping and Connectors

7.1 Per-channel 15-pin connectors

Table 11: 15-pin per-channel mapping (both input and output connectors)

Pin	Function
1	Routed signal: `DIRECT` or `FILTERED` per MOSWITCH
2–8, 10–15	Straight pass-through
9	Ground (GND)

7.2 Control header (uChip to board)

Table 12: Control nets

Net	Description
CTRL_A ... CTRL_L	Per-channel MOSWITCH control inputs
+5V	LED supply
GND	Common ground

8 Altium CAD Status and ECO/Compile Fix

8.1 Current status

Schematic hierarchy (12 channel sheets + LED blocks) is complete. Design rules are set; ground polygon exists. PCB update is blocked by:

- **Unknown Pin** on some capacitors/headers/transistors (symbol pin ↔ footprint pad mismatch).
- **Failed to add class member** (stale classes from earlier iterations).

8.2 Fix checklist (do in order)

1. **Synchronize sheet entries:** For each sheet symbol on the top sheet: *Sheet Symbol Actions* → *Synchronize Sheet Entries and Ports*. Delete any legacy *Port* objects from the top sheet; use Net Labels. *Compile*.
2. **Audit pin mapping:** Component → *Models* → *Edit Mapping*. Ensure symbol pin numbers match footprint pad numbers. Headers (double-row) are the usual culprit. BC337: confirm B/C/E vs. pad numbers (TO-92 and SOT-23 differ).
3. **Purge classes:** Disable auto class generation (Project Options). PCB: *Design* → *Classes...* → delete stale classes. Re-enable after a clean update.
4. **Final DRC:** Linked footprints (green), polygon repour, fix mask/silk violations, then generate outputs.

9 Bring-Up and Test Plan

9.1 Power-only checks

1. Visual inspection; continuity of GND and +5V rails; no shorts.
2. Power at 5 V; check MCU enumerates over USB; run LED walk test.

9.2 Selector verification

1. Drive CTRL_A high; confirm LED A on; scope the two MOSWITCH outputs (A/B) to verify selection behavior.
2. Apply a small step to the input; capture filtered output; confirm ~ 50 –100 ms settling.

9.3 With motor attached

1. Attach a motor to channel A; repeat step response; confirm expected attenuation and phase vs. direct path.

2. Repeat for a representative mirror and translation stage.

10 Bill of Materials (condensed)

Table 13: Key line items (one channel; multiply by 12 for total)

RefDes	Value/Part	Footprint	Notes
R1–R3	10 kΩ, 1%	0805	Filter resistors
R4	30 kΩ, 1%	0805	Direct path series
C1–C3	0.811 µF nominal	0805	Use 0.82 μF E12, X7R
Q1	BC337 or SOT-23 NPN	TO-92 / SOT-23	LED sink
R_B	3.3 kΩ	0805	Base resistor
R_LED	1.5 kΩ	0805	LED series
LED	Red, $\sim$ 2 mA	0805	Indicator
SW	MOSWITCH-SPDT	2.54 mm header	Selector module
J_IN/OUT	15-pin headers	2.54 mm	Per channel

11 Conclusion and Future Work

In summary, this project has successfully completed the foundational design for the CTN Slip–Stick Piezo Motor Filter/Selector Board. The three-stage RC filter was sized to achieve a 10 Hz corner frequency, and the design incorporates a direct path to ensure constant input loading. Key components like the MOSWITCH selector and the LED driver have been fully integrated into a comprehensive 12-channel schematic, and motor measurements were performed and documented to validate the filter's performance. The microcontroller communication protocol and a firmware skeleton have also been established and tested.

The primary task in progress involves resolving Altium CAD issues, specifically correcting pin mappings and cleaning up class definitions to prepare for final routing. Once these are complete, the next steps include a final design rule check (DRC) and ordering prototypes. Subsequent bring-up and testing will verify the core functionalities and performance with an attached motor. We will also decide on the capacitor values for the 5 Hz and 1 Hz variants, and finalize the firmware and host tools. This phased approach will ensure a robust and reliable system for production.

Appendix: Practical Tips

- **Capacitor choice.** Use X7R or C0G where size allows; verify actual capacitance at DC bias if motors see appreciable bias.

- **Layout.** Keep the three RC nodes compact; guard the filtered path from LED currents; star-ground the MOSWITCH return if possible.

- **Measurement.** For EPR, measure both polarities and average; for LCR, note fixture parasitics at 100 Hz.

- **Safety.** Respect MOSWITCH absolute maximum ratings; use current-limited bench supplies during bring-up.